STM32F407 Debug Commands Cheat Sheet

Oct 4, 2026 · @Arun R

Make commands

Run from the project folder. Only one program can use the ST-Link at a time: stop `make load` (Ctrl+C) before `make flash` or a VS Code debug session.

Command	What it does
`make`	Build: `build/final.elf` + `image/final.bin/.hex`
`make semi`	Semihosting build (printf to OpenOCD): `build/final_sh.elf`
`make clean`	Delete `build/` and `image/`
`make flash`	Build, then program + verify + reset the board, and exit
`make load`	Start OpenOCD and wait for GDB on port 3333 (does not flash)
`make -n <target>`	Show the commands without running them

Start a GDB session

Terminal 1 (leave running until you see `Listening on port 3333 for gdb connections`):

```
make load
```

Terminal 2:

```
arm-none-eabi-gdb build/final.elf
```

Inside GDB:

```
target extended-remote localhost:3333
monitor reset halt
load
break main
continue
```

After a code change: `make` in a terminal, then in GDB:

```
load
monitor reset halt
```

VS Code version (no terminals needed):

1. Stop `make load` if it's running (Ctrl+C).
2. Cmd+Shift+D → pick **Debug (OpenOCD)** → F5.
3. It builds, flashes and stops at `main()`. Step with F10 / F11, continue with F5.
4. After a code change: Shift+F5 (stop), then F5 again. That rebuilds and reflashes. Restart (Cmd+Shift+F5) only resets the chip.

To use an OpenOCD you already started with `make load`, pick **Attach to running OpenOCD (make load)** instead.

GDB commands

Command	Short	What it does
`next`	`n`	Next line, without going into function calls
`step`	`s`	Next line, going into function calls
`finish`	—	Run until the current function returns
`continue`	`c`	Run until the next breakpoint
Ctrl+C	—	Pause the running program
`break button_init`	`b`	Breakpoint on a function
`break main.c:36`	`b`	Breakpoint on a line
`info breakpoints`	`i b`	List breakpoints
`delete 1`	`d 1`	Remove breakpoint 1 (`delete` alone removes all)
`print RCC->AHB1ENR`	`p`	Show a variable or register
`p/x RCC->AHB1ENR`	—	Same, in hex
`x/4xw 0x40023830`	—	Show 4 words of memory, in hex, at an address
`info registers`	`i r`	Show CPU registers
`bt`	—	Show which functions called the current one
`list`	`l`	Show source around the current line

Command	Short	What it does
`monitor reset halt`	—	Reset the chip and stop it
`load`	—	Flash the current ELF file
`quit`	`q`	Exit GDB

Semihosting (printf to OpenOCD)

Add `printf("...\n");` after `initialise_monitor_handles()` inside `#ifdef SEMIHOSTING`. Output appears in the OpenOCD terminal, only when a line ends with `\n`.

Terminal 1:

```
make semi
make load
```

Terminal 2:

```
arm-none-eabi-gdb build/final_sh.elf
```

Inside GDB:

```
target extended-remote localhost:3333
monitor reset halt
monitor arm semihosting enable
load
continue
```

VS Code version:

1. Stop `make load` if it's running (Ctrl+C).
2. Cmd+Shift+D → pick **Debug semihosting (OpenOCD)** → F5. It runs `make semi` and turns semihosting on for you.
3. Press F5 again to run past `main()`.
4. `printf` output appears in the Terminal panel, tab **gdb-server**.

The semihosting build only runs with the debugger attached. Use `make` / `make flash` for normal runs.

VS Code debugging

Cmd+Shift+D → pick a configuration → F5.

Configuration	Needs
Debug (OpenOCD)	OpenOCD **not** running
Attach to running OpenOCD (make load)	`make load` running in a terminal
Debug semihosting (OpenOCD)	OpenOCD **not** running; output in the "gdb-server" terminal tab

Key	Action
F5	Start / continue
F10	Step over
F11	Step into
Shift+F11	Step out
F6	Pause
Cmd+Shift+F5	Restart
Shift+F5	Stop
Cmd+Shift+B	Build (`make` task)

GDB commands in the Debug Console need an `-exec` prefix, e.g. `-exec monitor reset halt`, `-exec p/x RCC->AHB1ENR`.

Quick checks

Check	Command
Is OpenOCD running?	`pgrep -fl openocd`
Stop a leftover OpenOCD	`pkill openocd`
Is it listening for GDB?	`lsof -nP -iTCP:3333 -sTCP:LISTEN`
Is the ST-Link on USB?	`system_profiler SPUSBDataType | grep -i -A3 st-link`
Flash through a running OpenOCD	`nc localhost 4444`, then `program /full/path/build/final.elf verify reset`

OpenOCD ports: 3333 = GDB, 4444 = commands, 6666 = Tcl.